

Name: Prakruti Prasanna Bhat

SRN: PES1UG24CS330

Lab 3: Component Modelling & Architectural Pattern Selection

PROBLEM STATEMENT #26 | Smart Cities, Transport & Logistics

Peer-to-Peer Carpooling & Expense Splitter

Architecture Selection:

We chose **Layered Architecture** for the **Peer-to-Peer Carpooling & Expense Splitter System**.

This system is suitable for Layered Architecture because it separates the application into different layers: Presentation Layer, Business Layer, Data Layer. The System Design includes the Presentation Layer, which contains the User Interface Component; the Business Layer, which contains the Ride Management, Route Matching, Expense Split, and Verification & Rating Components; and the Data Layer, which contains the Database Component.

Layered Architecture is chosen for the following reasons. The first reason is clear separation of responsibilities. Each element is responsible for one function. For instance, the Route Matching Component checks route compatibility and detour limits, and the Expense Split Component decides the rider's share of fuel costs. This makes the system easier to understand, test, and maintain.

The second reason is modifiability. You can change one part of the system, with fewer knock-on effects elsewhere. For example, the route-matching logic can be upgraded without changing the user interface or database component.

From the security point of view, an advantage of this architecture is that sensitive data such as driving-license validation information, user ratings, and travel history can be retrieved through the Data Layer and not directly from the User Interface. This enables security controls like encryption and limited access to sensitive data to be implemented. Your previous requirement does say this information should be stored securely with encryption and encrypted backups specifically.

The performance advantage is that each business component performs one specific task. This reduces unnecessary processing between different parts of the system. For instance, the Route Matching Component can concentrate on searching compatible rides efficiently. This confirms the need to return results of compatible rides within 3 seconds for the defined workload.